WhatsApp Pain-Point Discovery Bot

Functional Specification & Flow Design

Prepared for: AI Agents Training Assignment | Date: August 13, 2026

1. Overview

This bot runs on WhatsApp and is entered via a QR code that opens a pre-filled WhatsApp deep link ("Hello Oscar"). Its job is to identify the customer, ask about a business pain point they have, validate that the response is a genuine, sufficiently detailed pain point using an AI reasoning step, acknowledge it, and offer to schedule a short discovery call. If the customer accepts, the bot negotiates a time and books a calendar event, sending a confirmation with an RSVP link back over WhatsApp.

The system has three main capability layers: a messaging layer (WhatsApp Business API via the "Hello Oscar" service), a reasoning layer (LLM-based classification/validation of pain points and free-text scheduling replies), and a tool layer (contact lookup/CRM, calendar booking).

2. Actors and Components

Component	Role
Customer	Initiates contact by scanning the QR code; supplies name, business, pain point, and availability.
Hello Oscar API	WhatsApp Business API integration layer. Sends/receives messages, routes inbound events to the bot logic.
Session/State Store	Tracks each conversation's current stage per phone number (new/returning, awaiting pain point, awaiting elaboration, awaiting meeting confirmation, etc.).
User/CRM Lookup	Looks up phone number to determine returning vs. new customer and retrieve prior context (name, business, past notes).
Pain Point Validation Agent	LLM step that checks (a) is this a genuine business pain point, and (b) are the details clear enough to act on.
Scheduling Agent	Parses free-text availability from the user, proposes/matches time slots, and resolves conflicts.
Calendar API	Creates the calendar event and generates the RSVP/invite link (e.g., Google Calendar).
Lead/CRM Store	Persists captured pain points, qualification outcome, and meeting status for sales follow-up.

3. Conversation Flow (State Machine)

States are listed in order; transitions describe the triggering condition.

3.1 Entry

- S0 - QR Scanned: user scans QR code -> WhatsApp opens with prefilled text "Hello Oscar" -> message sent to Hello Oscar API.

3.2 Identification

- S1 - Lookup: bot looks up the sender's phone number against the CRM.
- S1a - Returning user -> personalized greeting using stored name/context.

- S1b - New user -> welcome message, bot asks for name and business name; user reply is captured and stored.

3.3 Pain Point Capture

- S2 - Ask: bot asks if the user has a business pain point AI could help address.
- S3 - Submit: user sends free-text response.
- S4 - Validate (AI step): checks (1) is this actually a business pain point, and (2) are the details clear.
- S4a - Fails either check -> bot replies "Please elaborate" -> returns to S3 (loop until valid, capped e.g. 3 attempts before falling back to a human handoff).
- S4b - Passes both checks -> proceeds to S5.

3.4 Acknowledgment and Meeting Ask

- S5 - Acknowledge: bot confirms it understood the pain point and asks if the user can spare ~30 minutes to discuss further.
- S5a - User declines -> bot thanks them, logs the pain point in the CRM for later outbound follow-up, session ends politely.
- S5b - User accepts -> proceeds to S6.

3.5 Scheduling

- S6 - Propose: bot suggests a date/time (or asks for the user's preference) for a 30-minute "AI Labs" catchup.
- S7 - User shares availability (free text).
- S8 - Match/Confirm: scheduling agent checks the proposed time against calendar availability.
- S8a - Conflict / doesn't work -> bot proposes an alternative -> returns to S6/S7 loop.
- S8b - Time confirmed -> bot books the calendar event via the Calendar API.
- S9 - Confirmation: bot sends a message with event details and an RSVP link.
- S10 - End: session marked complete; lead status updated to "meeting scheduled" in CRM.

4. Data Model (minimum viable fields)

Entity	Key Fields
Contact	phone_number (PK), name, business_name, first_seen_at, last_seen_at
Conversation Session	phone_number, current_state, attempt_count (elaboration loop), last_updated_at
Pain Point	id, phone_number, raw_text, is_valid (bool), validation_notes, captured_at
Meeting	id, phone_number, pain_point_id, proposed_time, confirmed_time, calendar_event_id, rsvp_link, status

5. AI / Agent Steps in Detail

5.1 Pain Point Validation

Input: free-text message from user. Output: structured judgment, e.g. { is_pain_point: bool, is_clear: bool, reason: string }. Suggested approach: a single LLM call with a rubric prompt ("Given this WhatsApp message, is the user describing a genuine business problem, and is it specific enough to act on? Reply in JSON."). Ambiguous or off-topic replies route back to the elaboration prompt rather than silently failing.

5.2 Scheduling Parser

Input: free-text availability (e.g., "Tuesday afternoon works" or "anytime after 3pm Thursday"). Output: a normalized date/time candidate to check against calendar availability. This is a natural-language-to-structured-time extraction task, ideally with a fallback that asks for an explicit date and time if parsing confidence is low.

6. External Integrations Required

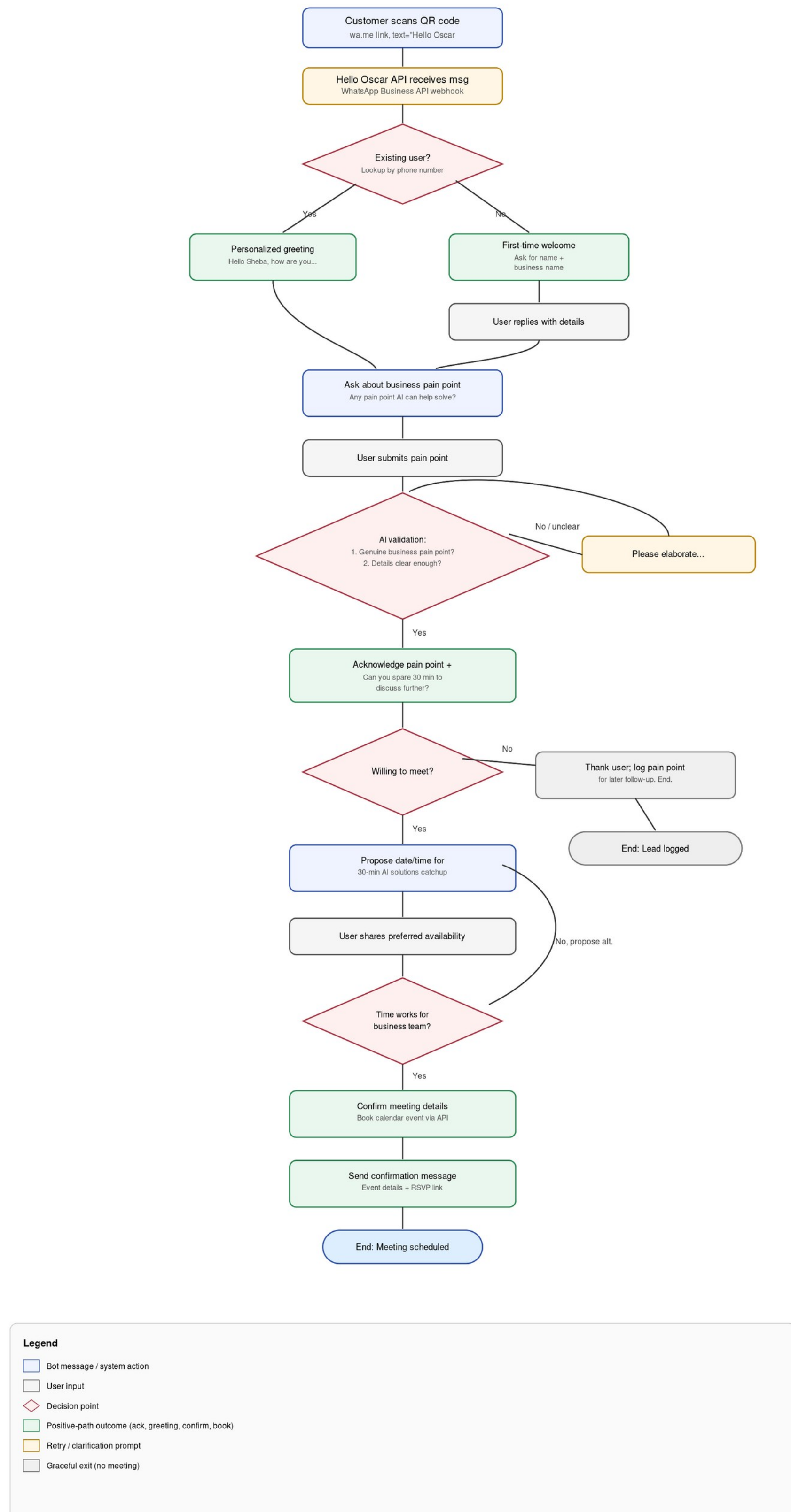
- WhatsApp Business API (via the Hello Oscar service) - send/receive messages, webhook for inbound events.
- CRM / database - contact lookup, pain point storage, lead status tracking.
- Calendar API (e.g., Google Calendar) - create events, generate invite/RSVP links, check availability.
- LLM provider - pain point validation and (optionally) scheduling text parsing.

7. Edge Cases and Open Questions

- Elaboration loop needs a cap (e.g., 3 attempts) with a graceful human-handoff fallback if the user still can't articulate a clear pain point.
- What happens if the user goes silent mid-flow (no timeout/re-engagement logic shown in the original diagram)?
- Multiple pain points in one session - does the bot capture one and move to scheduling, or allow more than one before asking about a meeting?
- Time zone handling for scheduling, especially since the example references Goa.
- Reschedule/cancel flow after a meeting is already confirmed.

8. Flowchart

The diagram below reflects the flow above, including the acknowledgment + "can you spare time" step after a valid pain point is captured.


Customer scans QR code
wa.me link, text="Hello Oscar
Hello Oscar API receives msg
WhatsApp Business API webhook
Existing user?
Lookup by phone number
Yes
No
Personalized greeting
Hello Sheba, how are you...
First-time welcome
Ask for name +
business name
User replies with details
Ask about business pain point
Any pain point AI can help solve?
User submits pain point
AI validation:
1. Genuine business pain point?
2. Details clear enough?
No / unclear
Please elaborate...
Yes
Acknowledge pain point +
Can you spare 30 min to
discuss further?
Willing to meet?
No
Thank user; log pain point
for later follow-up. End.
End: Lead logged
Yes
Propose date/time for
30-min AI solutions catchup
User shares preferred availability
No, propose alt.
Time works for
business team?
Yes
Confirm meeting details
Book calendar event via API
Send confirmation message
Event details + RSVP link
End: Meeting scheduled
Legend
Bot message / system action
User input
Decision point
Positive-path outcome (ack, greeting, confirm, book)
Retry / clarification prompt
Graceful exit (no meeting)